

Exception Handling in Python

1. What is an Exception?

An **exception** is an error that occurs while a program is running. If an exception is not handled, the program stops unexpectedly.

Example

```
a = 10
b = 0

print(a / b)
```

Output:

```
ZeroDivisionError: division by zero
```

The program crashes because division by zero is not allowed.

2. What is Exception Handling?

Exception handling is a mechanism used to handle runtime errors so that the program can continue or terminate gracefully.

Python provides these keywords:

- try
- except
- else
- finally
- raise

3. try and except

The code that may generate an exception is written inside the `try` block. The code used to handle the exception is written inside the `except` block.

Syntax

```
try:
    # risky code
except:
    # error handling code
```

Example

```
try:
    a = 10
    b = 0
    print(a / b)

except:
    print("An error occurred")
```

Output:

An error occurred

4. Handling Specific Exceptions

It is better to specify the type of exception.

```
try:
    num = int(input("Enter a number: "))
    print(10 / num)

except ZeroDivisionError:
    print("Cannot divide by zero")

except ValueError:
    print("Please enter a valid number")
```

Common Python Exceptions

Exception	Meaning
ZeroDivisionError	Division by zero
ValueError	Invalid value
TypeError	Wrong data type
IndexError	Invalid list index
KeyError	Dictionary key not found
FileNotFoundError	File does not exist
NameError	Variable is not defined

5. Catching the Exception Message

We can store the exception in a variable using `as`.

```
try:
    print(10 / 0)

except ZeroDivisionError as e:
    print("Error:", e)
```

Output:

Error: division by zero

6. Multiple except Blocks

A `try` block can have multiple `except` blocks.

```
try:
    num = int(input("Enter a number: "))
    result = 100 / num
    print(result)

except ValueError:
    print("Invalid input")

except ZeroDivisionError:
    print("Number cannot be zero")
```

7. The else Block

The `else` block executes **only when no exception occurs**.

```
try:
    num = int(input("Enter a number: "))

except ValueError:
    print("Invalid input")

else:
    print("You entered:", num)
```

Flow

No Exception → `else` executes
Exception → `except` executes

8. The finally Block

The `finally` block executes **whether an exception occurs or not**.

```
try:
    print(10 / 2)

except ZeroDivisionError:
    print("Cannot divide by zero")

finally:
    print("Program completed")
```

`finally` is commonly used for cleanup operations, such as closing files or database connections.

9. The raise Keyword

The `raise` keyword is used to manually generate an exception.

```
age = int(input("Enter your age: "))

if age < 0:
    raise ValueError("Age cannot be negative")

print("Age:", age)
```

10. Custom Exception

You can create your own exception class.

```
class InvalidAgeError(Exception):
    pass

age = int(input("Enter age: "))

if age < 18:
    raise InvalidAgeError("Age must be 18 or above")
```